

作业4

1.

下面的程序采用 Pthreads API。请完成该程序的编译链接和运行（提示：使用 pthread 库），截图给出编译链接此程序的命令和运行结果。并给出此程序的 LINE C 和 LINE P 的输出结果的分析。

```
#include <pthread.h>
#include <stdio.h>

int value = 0;
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
    pid_t pid;
    pthread_t tid;
    pthread_attr_t attr;

    pid = fork();

    if (pid == 0) { /* child process */
        pthread_attr_init(&attr);
        pthread_create(&tid,&attr,runner,NULL);
        pthread_join(tid,NULL);
        printf("CHILD: value = %d",value); /* LINE C */
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINE P */
    }
}

void *runner(void *param) {
    value = 5;
    pthread_exit(0);
}
```

- line c: child: value = 5
- line p parent: value = 0

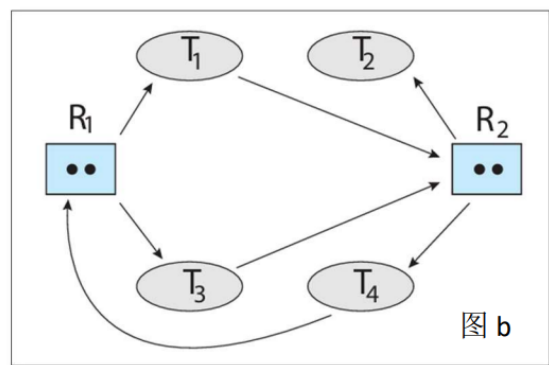
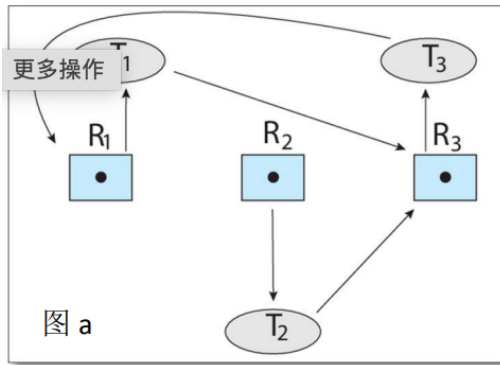
2.

PPT 中给出的读者-写者问题的解决方案，是否会导致饥饿现象？如果会，请列举一种可能会产生饥饿现象的场景。

PPT上是第一读者-写者问题，如果一个读者正在读，其他读者源源不断到来，写作者会饥饿

4.

下面的两个图中，请列出每个进/线程当前的状态（状态 1：就绪/运行，或者状态 2：等待）。哪个(些)图中的系统正处于死锁状态？如果处于死锁状态，请列出处于死锁的线程和资源形成的环。如果不是处于死锁状态，请列出可能顺利完成执行的线程顺序。



- 图a: T1 已分配 R1 申请 R3, T3 已分配 R3 申请 R1, T2 已分配 R2 申请 R3, 资源数量为一且有环 R1->T1->R3->T3->R1, 死锁
- 图b: T1 已分配 R1 申请 R2, T2 已分配 R2, T3 已分配 R1 申请 R2, T4 已分配 R2 申请 R1。运行顺序: T2->T1 (或T3) ->... (不止一种顺序)

5.

死锁 PPT 第 42 页, 两个问题中选择一个作为本题。起始状态为完成(1,0,2)的分配之后。

PPT: Can request for (3,3,0) / (0,2,0) by P0 be granted?

起始状态为完成(1, 0, 2)分配后:

	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	2	3	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

(3, 3, 0): Available A 不足, 无法分配

(0, 2, 0): 分配后的状态为:

	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	3	0	7	2	3	2	1	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

此时无法找到安全序列, 故不能分配